

AI Video Caption Generator

PROJECT NOTES — v1.0.0

Nekkanti Satya Srinath

1. Project Overview

AI Video Caption Generator is a Python and Streamlit application designed to generate multilingual captions from videos and create a separate video with captions permanently burned into it.

2. Core Objective

The end-to-end local-first caption workflow is:

- Upload a video.
- Preserve the original video.
- Generate timestamped transcription using Whisper.
- Detect the spoken language.
- Select the target caption language.
- Translate caption segments using local Ollama/TranslateGemma.
- Generate SRT and VTT caption files.
- Use FFmpeg to permanently burn captions into a separate output video.

3. Technology Stack

- Python 3.11
- Streamlit
- OpenAI Whisper
- Ollama
- TranslateGemma 12B
- FFmpeg
- PyTest
- GitHub Actions
- Local file storage

4. Application Workflow

Upload → Save Original → Whisper Transcription → Language Detection → Caption Language Selection → Local Translation → SRT/VTT → FFmpeg Caption Burn → Captioned Video

5. Output Structure

- uploads/ — original uploaded videos
- captions/ — generated SRT and VTT files

- outputs/ — captioned videos

6. Supported Video Formats

MP4, MOV, AVI, MKV and WebM.

7. Supported Caption Languages

- English (en)
- Telugu (te)
- Hindi (hi)
- Tamil (ta)
- Kannada (kn)
- Malayalam (ml)
- Bengali (bn)
- Marathi (mr)
- Gujarati (gu)
- Punjabi (pa)

8. Caption Formats

- SRT — SubRip subtitle format.
- VTT — WebVTT subtitle format.

9. Architecture Notes

The application follows a modular architecture with Streamlit UI, agents for workflow coordination, services for reusable processing, providers for replaceable integrations, core models, configuration and utility modules.

- Streamlit UI
- Pages / Agents
- Services
- Providers
- Whisper / Ollama / FFmpeg

10. Important Design Decisions

- Local-first AI processing for the core translation workflow.
- Original videos are preserved and are not overwritten.
- Generated videos are stored separately under outputs/.
- Translation is abstracted behind TranslationProvider.
- Transcript data is used internally during caption processing rather than maintained as transcript history.
- SRT and VTT are both generated as standard subtitle outputs.
- Dashboard excludes .gitkeep placeholder files from recent files.

11. UI Pages

- Dashboard
- Caption Generator
- Captions
- Settings
- Help
- About
- Sidebar navigation

12. Testing Status

The final automated test suite was executed successfully on Windows PowerShell using Python 3.11.4 and PyTest 9.1.1.

Metric	Result
Total tests	153
Passed	153
Failed	0
Pass rate	100%
Execution time	15.31 seconds
Final status	TESTED SUCCESSFULLY

13. Exact Verified Test Result

```
153 passed in 15.31s
```

```
153 tests
153 passed
0 failed
100% pass rate
TESTED SUCCESSFULLY
```

14. End-to-End Verification

- Original video saved successfully.
- SRT generated successfully.
- VTT generated successfully.
- Captioned video generated successfully.
- Dashboard verified.
- Settings verified.
- Help verified.
- About verified.
- Sidebar navigation verified.
- GitHub Actions CI configured.

15. Documentation Status

The project documentation set includes README, architecture, system design, configuration, installation, testing, troubleshooting, user guide, workflow, provider guide, interview material, resume bullets, release notes, changelog, security and contribution guidance.

16. Release Status

v1.0.0 — Core Release Complete

- 153 automated tests passed.
- 0 automated test failures.
- 100% test pass rate.
- End-to-end media workflow verified.
- Original, SRT, VTT and captioned-video outputs verified.

17. Future Enhancements

- Caption editor.
- Batch video processing.
- Caption styling controls.
- Additional translation providers.
- Additional subtitle formats.
- Advanced FFmpeg controls.

18. Author

Nekkanti Satya Srinath

19. License

The project is released under the **MIT License**.